SKILL-3

TASK-1

hpharipriya@LAPTOP-M4TV0

GNU nano 8.7.1 main.c *

```c
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    while (1) {
        printf("mysh> ");

        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0) {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

```
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ nano main.c
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ gcc main.c
-o main
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ ./main
mysh> HELLO
You entered: HELLO
mysh> HARI PRIYA
You entered: HARI PRIYA
mysh> exit
Exiting shell...
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$
```

TASK-2

hpharipriya@LAPTOP-M4TV0

GNU nano 8.7.1 copy.c *

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0) {
        printf("Error opening File1.txt\n");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0) {
        printf("Error creating File2.txt\n");
        close(fd1);
        return 1;
    }

    while ((n = read(fd1, buffer, sizeof(buffer))) > 0) {
        write(fd2, buffer, n);
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully!\n");

    return 0;
}
```

```
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ nano File1.
txt
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ nano copy.c
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ gcc copy.c
-o copy
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ ./copy
File copied successfully!
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ cat File2.t
xt
Hello! This is my Skill Week 3 Task 2.

hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ ls
File1.txt  File2.txt  copy  copy.c  main  main.c
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$
```

TASK-3

hphariprya@LAPTOP-M4TV0

GNU nano 8.7.1 input.c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    char buffer[100];
    int index = 0;
    char ch;

    printf("mysh> ");
    fflush(stdout);

    while (1) {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n') {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            if (index == 4 &&
                buffer[0] == 'e' &&
                buffer[1] == 'x' &&
                buffer[2] == 'i' &&
                buffer[3] == 't') {
                printf("Exiting shell...\n");
                break;
            }

            index = 0;
            printf("mysh> ");
            fflush(stdout);
        }
        else if (ch == 127 || ch == '\b') {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
```

```c
                printf("\b \b");
                fflush(stdout);
            }
        }
        else if (index < 99) {
            buffer[index++] = ch;
            write(STDOUT_FILENO, &ch, 1);
        }
    }

    return 0;
}
```

```
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ gcc input.c
 -o input
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-01$ ./input
mysh> hello world
hello world
You entered: hello world
mysh> HELLO WRLD
HELLO WRLD
You entered: HELLO WRLD
mysh> HELLO WORLD
HELLO WORLD
You entered: HELLO WORLD
mysh> exit
exit
You entered: exit
Exiting shell...
```

TASK-4



```
        } else {
            for (int i = 1; i < MAX_HISTORY; i++) {
                strcpy(history[i - 1], history[i]);
            }
            strcpy(history[MAX_HISTORY - 1], command);
        }

        printf("Command stored: %s\n", command);

        /* Display history */
        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < history_count - 1; i++) {
                printf("%d: %s\n", i + 1, history[i]);
            }
        }
    }

    return 0;
}
```

```
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ nano histor
y.c
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ gcc history
.c -o history
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ ./history
mysh> ls
Command stored: ls
mysh> pwd
Command stored: pwd
mysh> hello
Command stored: hello
mysh> history
Command stored: history

Command History:
1: ls
2: pwd
3: hello
mysh> exit
Exiting shell...
```

hpharipriya@LAPTOP-M4TV0

```
 GNU nano 8.7.1                          history.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

void enable_raw_mode(struct termios *old) {
    struct termios new;

    tcgetattr(STDIN_FILENO, old);
    new = *old;

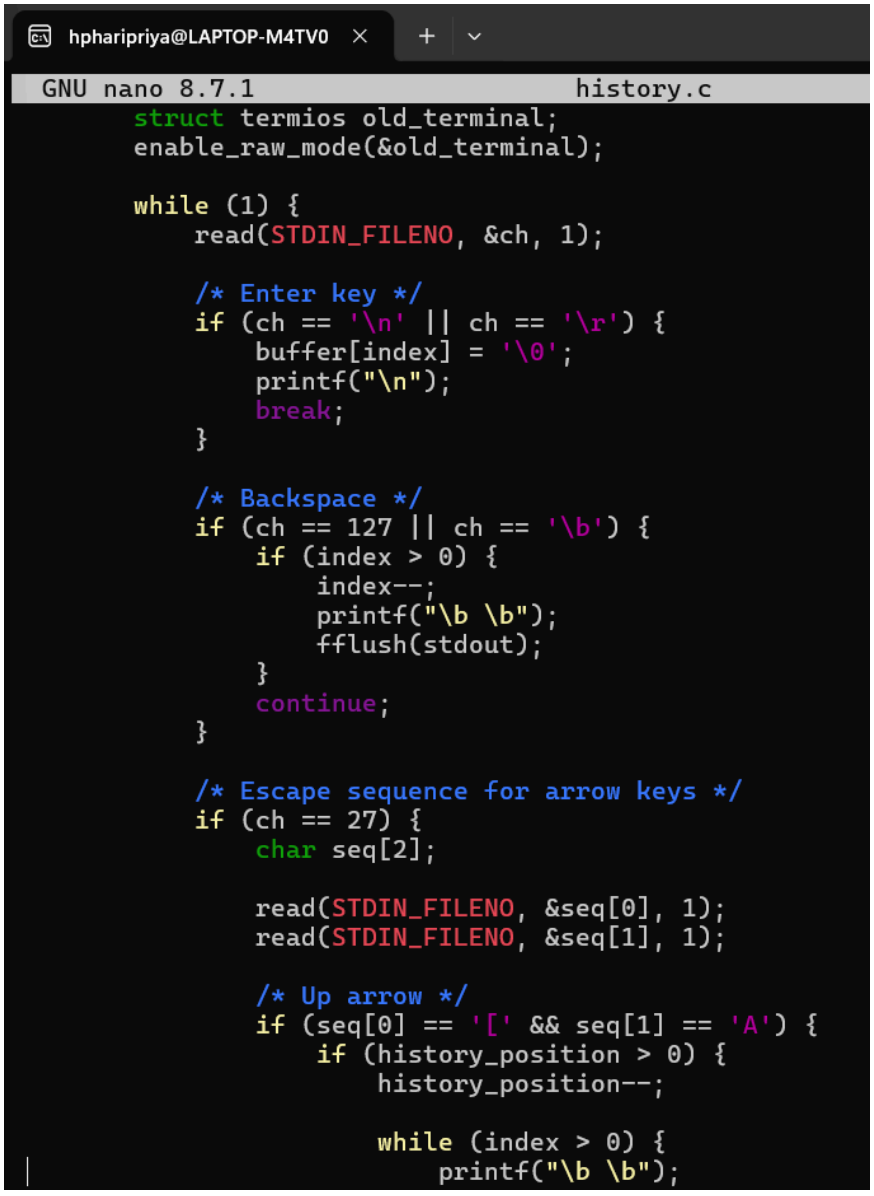
    new.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &new);
}

void disable_raw_mode(struct termios *old) {
    tcsetattr(STDIN_FILENO, TCSANOW, old);
}

int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    int history_count = 0;
    char buffer[MAX_LENGTH];
    int index;
    int history_position;
    char ch;

    while (1) {
        printf("mysh> ");
        fflush(stdout);

        index = 0;
        history_position = history_count;
```


hpharipriya@LAPTOP-M4TV0
GNU nano 8.7.1 history.c
struct termios old_terminal;
enable_raw_mode(&old_terminal);
while (1) {
read(STDIN_FILENO, &ch, 1);
/* Enter key */
if (ch == '\n' || ch == '\r') {
buffer[index] = '\0';
printf("\n");
break;
}
/* Backspace */
if (ch == 127 || ch == '\b') {
if (index > 0) {
index--;
printf("\b \b");
fflush(stdout);
}
continue;
}
/* Escape sequence for arrow keys */
if (ch == 27) {
char seq[2];
read(STDIN_FILENO, &seq[0], 1);
read(STDIN_FILENO, &seq[1], 1);
/* Up arrow */
if (seq[0] == '[' && seq[1] == 'A') {
if (history_position > 0) {
history_position--;
while (index > 0) {
printf("\b \b");


hpharipriya@LAPTOP-M4TV0
GNU nano 8.7.1 history.c
index--;
}
strcpy(buffer, history[history_p
index = strlen(buffer);
printf("%s", buffer);
fflush(stdout);
}
}
continue;
}
/* Normal character */
if (index < MAX_LENGTH - 1) {
buffer[index++] = ch;
write(STDOUT_FILENO, &ch, 1);
}
}
disable_raw_mode(&old_terminal);
/* Exit condition */
if (strcmp(buffer, "exit") == 0) {
printf("Exiting shell...\n");
break;
}
/* Ignore empty commands */
if (strlen(buffer) == 0) {
continue;
}
/* Store command */
if (history_count < MAX_HISTORY) {
strcpy(history[history_count], buffer);


/* Store command */
if (history_count < MAX_HISTORY) {
strcpy(history[history_count], buffer);
history_count++;
} else {
for (int i = 1; i < MAX_HISTORY; i++) {
strcpy(history[i - 1], history[i]);
}
strcpy(history[MAX_HISTORY - 1], buffer);
}
printf("Command stored: %s\n", buffer);
}
return 0;

```
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ nano histor
y.c
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ gcc history
.c -o history
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$ ./history
mysh> hello
Command stored: hello
mysh> world
Command stored: world
mysh> testing
Command stored: testing
mysh> testing
Command stored: testing
mysh> EXIT
Command stored: EXIT
mysh> exit
Exiting shell...
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-04$
```

TASK-5

hpharipriya@LAPTOP-M4TV0

```
 GNU nano 8.7.1                                memory.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    int capacity = 2;
    int count = 0;

    char **commands = malloc(capacity * sizeof(char *));

    if (commands == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Dynamic Memory and Linked List Demo\n");

    /* Add commands dynamically */
    const char *input[] = {
        "ls",
        "pwd",
        "gcc main.c -o main",
        "echo Hello"
    };

    int total = 4;

    for (int i = 0; i < total; i++) {

        /* Resize when capacity is full */
        if (count >= capacity) {
            capacity *= 2;
```

hpharipriya@LAPTOP-M4TV0

GNU nano 8.7.1 memory.c *

```
            char **temp = realloc(commands,
                                  capacity * sizeof(char *));

            if (temp == NULL) {
                printf("Memory reallocation failed.\n");

                for (int j = 0; j < count; j++) {
                    free(commands[j]);
                }

                free(commands);
                return 1;
            }

            commands = temp;
            printf("Buffer resized to capacity: %d\n", capacity);
        }

        commands[count] = malloc(strlen(input[i]) + 1);

        if (commands[count] == NULL) {
            printf("Memory allocation failed.\n");

            for (int j = 0; j < count; j++) {
                free(commands[j]);
            }

            free(commands);
            return 1;
        }

        strcpy(commands[count], input[i]);
        count++;
    }

    printf("\nStored Commands:\n");
```

hpharipriya@LAPTOP-M4TV0

GNU nano 8.7.1 memory.c *

```
        printf("%d: %s\n", i + 1, commands[i]);
    }

    /* Create a linked list */
    Node *head = NULL;
    Node *tail = NULL;

    for (int i = 0; i < count; i++) {

        Node *new_node = malloc(sizeof(Node));

        if (new_node == NULL) {
            printf("Linked list allocation failed.\n");
            break;
        }

        new_node->command = malloc(strlen(commands[i]) + 1);

        if (new_node->command == NULL) {
            free(new_node);
            break;
        }

        strcpy(new_node->command, commands[i]);
        new_node->next = NULL;

        if (head == NULL) {
            head = new_node;
            tail = new_node;
        } else {
            tail->next = new_node;
            tail = new_node;
        }
    }

    printf("\nLinked List:\n");
```

hpharipriya@LAPTOP-M4TV0

GNU nano 8.7.1 memory.c *

```
            tail = new_node;
        }
    }

    printf("\nLinked List:\n");

    Node *current = head;

    while (current != NULL) {
        printf("%s -> ", current->command);
        current = current->next;
    }

    printf("NULL\n");

    /* Free linked list memory */
    current = head;

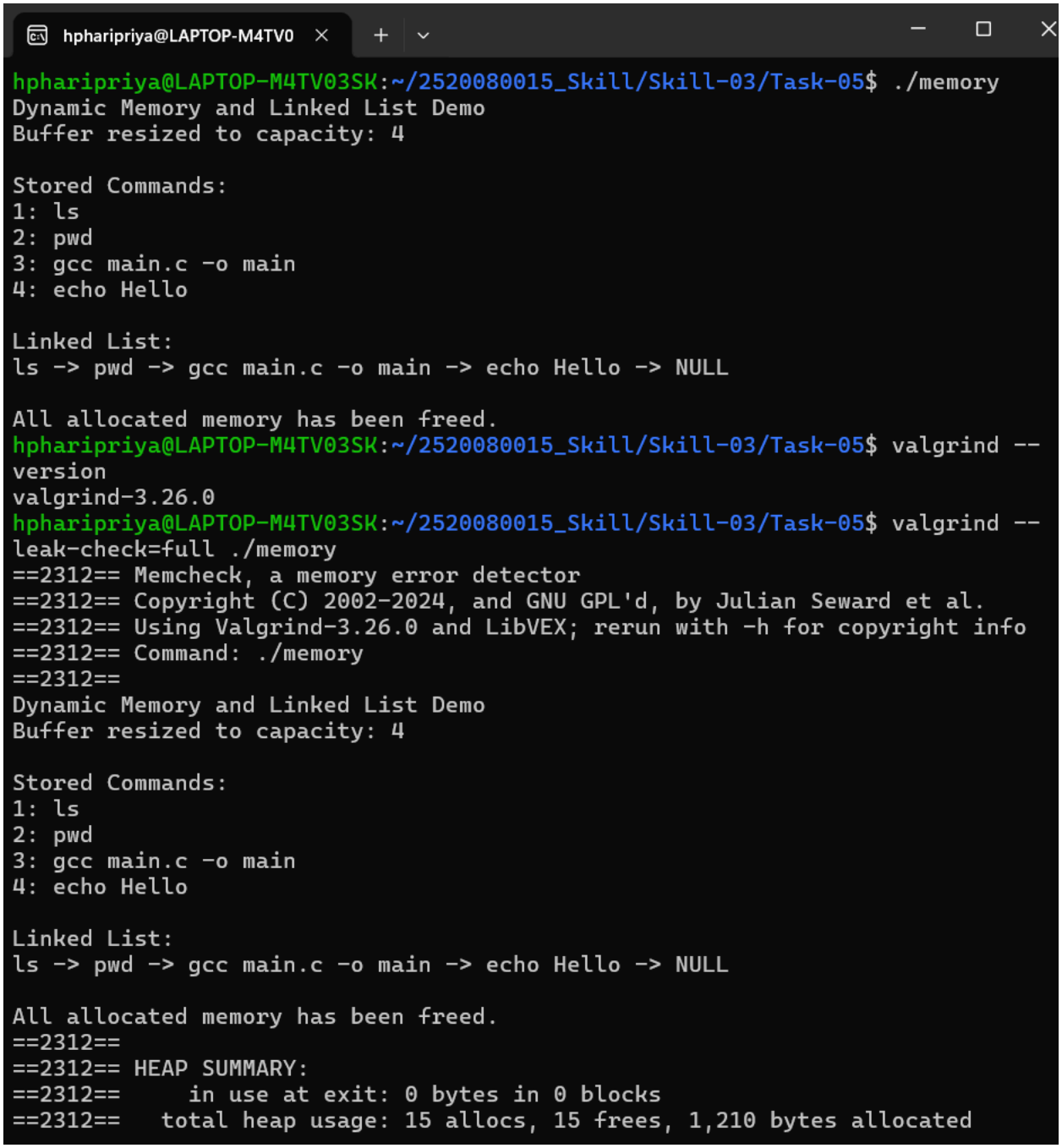
    while (current != NULL) {
        Node *next = current->next;
        free(current->command);
        free(current);
        current = next;
    }

    /* Free command array */
    for (int i = 0; i < count; i++) {
        free(commands[i]);
    }

    free(commands);

    printf("\nAll allocated memory has been freed.\n");

    return 0;
}
```


hpharipriya@LAPTOP-M4TV0
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-05$ ./memory
Dynamic Memory and Linked List Demo
Buffer resized to capacity: 4
Stored Commands:
1: ls
2: pwd
3: gcc main.c -o main
4: echo Hello
Linked List:
ls -> pwd -> gcc main.c -o main -> echo Hello -> NULL
All allocated memory has been freed.
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-05$ valgrind --version
valgrind-3.26.0
hpharipriya@LAPTOP-M4TV03SK:~/2520080015_Skill/Skill-03/Task-05$ valgrind --leak-check=full ./memory
==2312== Memcheck, a memory error detector
==2312== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2312== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2312== Command: ./memory
==2312==
Dynamic Memory and Linked List Demo
Buffer resized to capacity: 4
Stored Commands:
1: ls
2: pwd
3: gcc main.c -o main
4: echo Hello
Linked List:
ls -> pwd -> gcc main.c -o main -> echo Hello -> NULL
All allocated memory has been freed.
==2312==
==2312== HEAP SUMMARY:
==2312== in use at exit: 0 bytes in 0 blocks
==2312== total heap usage: 15 allocs, 15 frees, 1,210 bytes allocated